

Phân tích và nghiên cứu ứng dụng của thuật toán di truyền

Qian Ziqiang
Trường Trung học số 8 Fuzhou

2005

Nguồn gốc: Phân tích và nghiên cứu ứng dụng của thuật toán di truyền

IOI China candidate team papers / 2005 / Qian Ziqiang.doc

Tóm tắt

Cùng với sự phát triển của khoa học kỹ thuật, các bài toán gặp trong sản xuất và đời sống ngày càng phức tạp. Nhiều bài toán đòi hỏi tìm nghiệm tối ưu hoặc nghiệm xấp xỉ trong một không gian tìm kiếm rất lớn, nên việc dùng các thuật toán truyền thống trở nên khó khăn. Những năm gần đây, tư tưởng tiến hóa trong sinh học được ứng dụng rộng rãi trong kỹ thuật công trình, trí tuệ nhân tạo và nhiều lĩnh vực khác, tạo thành một lớp thuật toán tìm kiếm ngẫu nhiên hiệu quả gọi là thuật toán tiến hóa.

Bài viết trước hết giới thiệu nguyên lý và lịch sử của thuật toán tiến hóa, sau đó trình bày chi tiết các bước dùng thuật toán di truyền để giải bài toán, đồng thời nêu một số đề xuất cải tiến và kinh nghiệm áp dụng. Tiếp theo, bài viết dùng một ví dụ tối ưu hóa NP-khó để minh họa sâu hơn cách dùng thuật toán di truyền, rồi phân tích dữ liệu thử nghiệm để chứng minh hiệu quả của phương pháp.

Từ khóa: trí tuệ nhân tạo; thuật toán tiến hóa; thuật toán di truyền (GA); cây khung nhỏ nhất đa mục tiêu.

1 Lý thuyết thuật toán tiến hóa

1.1 Khái quát

Từ thời xa xưa, sự sống bắt đầu từ các sinh vật đơn bào, trải qua các biến đổi khắc nghiệt của môi trường và tiến hóa từ bậc thấp đến bậc cao, từ đơn giản đến phức tạp. Sự sống không ngừng sinh sôi, cuối cùng tạo ra các sinh vật có tư duy và trí tuệ. Con người có một cấu trúc sống ưu việt: không chỉ thụ động thích nghi với môi trường, mà còn có thể học tập, bắt chước và sáng tạo để liên tục nâng cao năng lực thích nghi.

Thuật toán tiến hóa là thuật toán tìm kiếm ngẫu nhiên mô phỏng chọn lọc tự nhiên và cơ chế di truyền. Nó dùng quy luật “ưu thắng liệt thải, thích nghi thì tồn tại” để khuyến khích các cấu trúc tốt, đồng thời dùng mô hình di truyền và biến dị kiểu Mendel để giữ lại cấu trúc đã có trong quá trình lặp và tìm kiếm cấu trúc tốt hơn. Là một phương pháp tối ưu hóa và tìm kiếm ngẫu nhiên, thuật toán tiến hóa có các đặc điểm sau: nó không tìm kiếm mù quáng, cũng không vét cạn toàn bộ không gian, mà dựa trên môi trường sống của cá thể, tức hàm mục tiêu, để tìm kiếm có định hướng. Thuật toán chỉ cần giá trị của mục tiêu, không cần thông tin khác, nên phù hợp với các bài toán tối ưu quy mô lớn, phi tuyến mạnh, không liên tục và có nhiều cực trị cục bộ; tính phổ dụng của nó rất cao. Đối tượng thao tác của thuật toán là một quần thể cá thể thay vì một cá thể đơn lẻ, vì thế nó có nhiều quỹ đạo tìm kiếm cùng lúc.

1.2 Thuật toán di truyền

Thuật toán di truyền (*Genetic Algorithm*) là một nhánh quan trọng của thuật toán tiến hóa. Nó do John Holland đề xuất, ban đầu dùng để nghiên cứu quá trình thích nghi của hệ tự nhiên và thiết kế phần mềm có tính tự thích nghi. Gần đây, thuật toán di truyền đã được ứng dụng thành công như một công cụ giải bài toán và tối ưu hóa, đồng thời ngày càng phổ biến.

Đặc điểm chính của thuật toán di truyền là chiến lược tìm kiếm theo quần thể và việc trao đổi thông tin giữa các cá thể trong quần thể. Về bản chất, nó mô phỏng quá trình học tập tổng thể của một quần thể gồm nhiều cá thể, trong đó mỗi cá thể biểu diễn một điểm nghiệm trong không gian tìm kiếm. Từ một quần thể ban đầu bất kỳ, thuật toán dùng các thao tác di truyền như chọn lọc, lai ghép và đột biến để làm quần thể tiến hóa qua từng thế hệ tới những vùng ngày càng tốt hơn của không gian tìm kiếm, cho đến khi đạt tới điểm nghiệm tối ưu.

So với các phương pháp tìm kiếm khác, ưu thế của thuật toán di truyền thể hiện ở vài điểm. Trước hết, nó ít bị kẹt trong cực trị cục bộ; ngay cả khi hàm thích nghi không liên tục hoặc không đều, thuật toán vẫn có xác suất lớn tìm được nghiệm tối ưu toàn cục. Thứ hai, do có tính song song tự nhiên, thuật toán di truyền rất phù hợp với xử lý song song và phân tán quy mô lớn. Ngoài ra, thuật toán dễ kết hợp với các kỹ thuật khác như mạng nơ-ron, suy luận mờ, hành vi hỗn độn và sự sống nhân tạo để tạo ra phương pháp giải bài toán có hiệu năng tốt hơn.

2 Thuật toán di truyền

2.1 Quy trình cơ bản

Một thuật toán di truyền chạy tuần tự thường thực hiện các bước sau:

1. Mã hóa bài toán cần giải; đặt $t := 0$.

2. Khởi tạo ngẫu nhiên quần thể

$$X(0) = \{x_1, x_2, \dots, x_M\}.$$

3. Với mỗi nhiễm sắc thể x_i trong quần thể hiện tại $X(t)$, tính độ thích nghi $F(x_i)$. Độ thích nghi thể hiện năng lực thích ứng của cá thể với môi trường và quyết định xác suất cá thể ấy được rút chọn trong các thao tác di truyền.

4. Áp dụng các toán tử di truyền lên $X(t)$ theo xác suất đã đặt trước để tạo quần thể thế hệ mới $X(t+1)$. Mục đích của các toán tử này là mở rộng phạm vi bao phủ của số cá thể hữu hạn, thể hiện tư tưởng tìm kiếm toàn cục.

5. Đặt $t := t + 1$, tức dùng thế hệ mới thay thế thế hệ cũ. Nếu chưa đạt điều kiện dừng đã định, quay lại bước 3.

2.2 Các yếu tố quan trọng

Lý thuyết mã hóa. Thuật toán di truyền cần dùng một phương pháp mã hóa nào đó để ánh xạ không gian nghiệm sang không gian mã, chẳng hạn chuỗi bit, số thực hoặc chuỗi có thứ tự. Cách biểu diễn này tương tự cấu trúc nhiễm sắc thể sinh học, nên để giải thích bằng lý thuyết di truyền và cũng để hiện thực các thao tác di truyền. Lý thuyết mã hóa là một trong các yếu tố quyết định hiệu quả của thuật toán. Mã hóa nhị phân là cách thường dùng nhất vì có nhiều mẫu toán tử và dễ cài đặt. Tuy nhiên, trong từng bài toán cụ thể, nếu chọn được dạng mã phù hợp với không gian nghiệm, ta có thể thao tác trực tiếp hơn trên kiểu hình của nghiệm, để đưa thêm thông tin heuristic liên quan, và thường đạt hiệu quả cao hơn mã hóa nhị phân.

Nhiệm sắc thể. Mỗi chuỗi mã tương ứng với một nghiệm cụ thể của bài toán, được gọi là một nhiệm sắc thể hoặc một cá thể. Thông qua phương pháp mã hóa đã chọn, một nhiệm sắc thể có thể tương ứng với một nghiệm cụ thể. Một số lượng nhiệm sắc thể cố định tạo thành một thể hệ quần thể; các nhiệm sắc thể trong quần thể có thể trùng nhau.

Khởi tạo ngẫu nhiên. Thể hệ đầu tiên có thể được sinh ngẫu nhiên hoặc sinh có quy luật, chẳng hạn từ một điểm đã biết gần nghiệm rời rạc, hoặc bằng cách phân bố đều các nghiệm khả thi. Trong một lần chạy thuật toán, việc chủ động khởi tạo quần thể nhiều lần có thể giúp thuật toán có năng lực tìm kiếm nghiệm tối ưu toàn cục mạnh hơn.

Độ thích nghi. Độ thích nghi của một nhiệm sắc thể là đánh giá năng lực sống sót của nó, và quyết định xác suất nhiệm sắc thể đó được rút chọn. Hàm đánh giá là tiêu chuẩn đo độ thích nghi. Các hàm thường gặp gồm: trực tiếp dùng trọng số của nghiệm, chẳng hạn với bài toán ba lô 0/1, dùng tổng giá trị đồ vật được đưa vào ba lô làm độ thích nghi; đếm số bit 1 hoặc 0 trong chuỗi nhị phân, với các thuật toán dùng mã nhị phân; hoặc cộng điểm của nhiệm sắc thể trên từng mục tiêu trong bài toán tối ưu đa mục tiêu. Với loại bài toán cuối, bài viết sẽ đưa ra một hàm đánh giá hiệu quả hơn.

Toán tử di truyền. Toán tử di truyền là phần lõi của thuật toán. Chúng tác động trực tiếp lên quần thể hiện có để sinh quần thể thế hệ kế tiếp, vì thế cách chọn và phối hợp toán tử, cũng như tỷ lệ của từng toán tử, ảnh hưởng trực tiếp đến hiệu quả của thuật toán. Một thuật toán di truyền thường gồm nhiều toán tử; mỗi toán tử chạy độc lập, còn bản thân thuật toán chỉ quy định tỷ lệ đóng góp của từng toán tử khi tạo thế hệ mới. Khi chạy, toán tử rút một số nhiệm sắc thể tương ứng từ quần thể hiện tại, xử lý chúng, rồi đưa nhiệm sắc thể mới vào thế hệ kế tiếp.

Một số toán tử di truyền thường gặp là:

- **Toán tử giữ lại:** rút một nhiệm sắc thể và đưa thẳng vào thế hệ kế tiếp. Toán tử này giúp thuật toán hội tụ.
- **Toán tử lai ghép:** rút hai nhiệm sắc thể, trao đổi một số đoạn giữa chúng, rồi chọn cá thể có độ thích nghi cao hơn, hoặc chọn cả hai, để đưa vào thế hệ kế tiếp. Toán tử này giúp thuật toán tận dụng các nghiệm đã có để tìm nghiệm tốt hơn.
- **Toán tử đột biến:** rút một nhiệm sắc thể, thay đổi ngẫu nhiên một phần của nó, rồi đưa vào thế hệ kế tiếp. Toán tử này giúp thuật toán thoát cực trị cục bộ, tăng năng lực tìm kiếm toàn cục và tránh rơi vào tìm kiếm cục bộ. Xác suất của toán tử không nên quá cao, nếu không nghiệm sẽ bị phân tán và thuật toán không thể hội tụ.

Rút chọn. Rút chọn là một thao tác cơ bản quan trọng. Nó chọn các nhiệm sắc thể từ quần thể hiện tại để cung cấp cho toán tử di truyền theo nguyên tắc “ưu thắng liệt thải”, dựa trên độ thích nghi của từng nhiệm sắc thể. Cách thường dùng là vòng quay có thiên lệch. Giả sử quần thể có M cá thể, trước hết tính tổng độ thích nghi

$$S = \sum_{i=1}^M F(x_i).$$

Chia các số nguyên từ 1 đến S thành M đoạn, trong đó đoạn của nhiệm sắc thể x_i có độ dài $F(x_i)$. Khi sinh ngẫu nhiên một số nguyên từ 1 đến S , ta chọn nhiệm sắc thể ứng với đoạn chứa số đó. Như vậy xác suất x_i được chọn là

$$p_i = \frac{F(x_i)}{S}.$$

Điều kiện dừng. Điều kiện dừng dùng để tránh cho thuật toán lặp mãi. Thông thường có thể dừng sau một số vòng lặp xác định, hoặc tự động dừng khi tốc độ hội tụ của nghiệm thấp hơn một ngưỡng nào đó. Khi đạt điều kiện dừng, thuật toán kết thúc và trả về nghiệm sắc thể tốt nhất thu được trong quá trình chạy, hoặc tất cả nghiệm tối ưu không bị trội nếu bài toán yêu cầu.

2.3 Thiết lập tham số quan trọng

Khi dùng thuật toán di truyền, ta thường phải chọn tham số khác nhau cho từng bài toán. Với các bài toán quy mô lớn, ngay trong một lần chạy cũng có thể đặt các tham số khác nhau cho những giai đoạn khác nhau. Các tham số ảnh hưởng lớn đến hiệu quả gồm:

- **Kích thước quần thể:** số nhiễm sắc thể trong một thế hệ. Quần thể càng lớn thì chứa được càng nhiều loại nhiễm sắc thể, có lợi cho việc tìm kiếm nghiệm tối ưu toàn cục, nhưng tốc độ hội tụ giảm và thời gian cần thiết tăng.
- **Số vòng lặp:** số lần cập nhật quần thể tối đa. Tăng số vòng lặp giúp nghiệm hội tụ chính xác hơn nhưng tốn nhiều thời gian hơn. Nếu thời gian cho phép, chạy nhiều lần với khởi tạo lại quần thể thường hiệu quả hơn đặt số vòng lặp rất lớn trong một lần chạy.
- **Tỷ lệ giữ lại:** tỷ lệ của toán tử giữ lại, thường không vượt quá 70%.
- **Tỷ lệ lai ghép:** tỷ lệ của toán tử lai ghép, thường không vượt quá 50%.
- **Tỷ lệ đột biến:** tỷ lệ của toán tử đột biến, thường không vượt quá 1%.

3 Ứng dụng vào cây khung nhỏ nhất đa mục tiêu

Nhiều bài toán đời sống như xây dựng đường sá, dựng mạng điện hay thiết kế mạng đều có thể quy về mô hình cây khung nhỏ nhất. Các thuật toán kinh điển như Prim và Kruskal đều giải được bài toán này với độ phức tạp tuyến tính theo số cạnh nếu đã có cấu trúc dữ liệu phù hợp. Tuy nhiên, bài toán thực tế thường không đơn giản như vậy. Một cạnh có thể mang nhiều trọng số: xây đường không chỉ xét chiều dài mà còn phải xét môi trường và yếu tố nhân văn; dựng mạng điện không chỉ xét chi phí dây mà còn xét độ khó thi công; nối mạng không chỉ xét độ trễ mà còn xét độ ổn định và an toàn truyền tải. Khi đó bài toán trở thành tìm các nghiệm tối ưu không bị trội của cây khung nhỏ nhất đa mục tiêu. Bài toán này là NP-khó, các thuật toán thông thường như Prim và Kruskal không còn đủ, còn tìm kiếm vét cạn thì quá đắt.

3.1 Bài toán cây khung nhỏ nhất đa mục tiêu

3.1.1 Cây khung nhỏ nhất

Trong lý thuyết đồ thị, một đồ thị con liên thông không có chu trình được gọi là cây. Gọi $T = (N, E_T)$ là một đồ thị với $|N| \geq 3$. Sáu mô tả sau về cây là tương đương:

1. T liên thông và không có chu trình.
2. T có $|N| - 1$ cạnh và không có chu trình.
3. T liên thông và có $|N| - 1$ cạnh.
4. T liên thông và mọi cạnh đều là cạnh cắt.
5. Giữa hai đỉnh bất kỳ của T có đúng một đường đi.

6. T không có chu trình, nhưng nếu thêm một cạnh giữa một cặp đỉnh không kề nhau thì tạo ra đúng một chu trình.

Xét đồ thị vô hướng

$$G = (V, E, w),$$

trong đó w là hàm trọng số. Nếu cây $T = (V, E_T)$ chứa tất cả đỉnh của G , thì T là một cây khung của G , có trọng số

$$W(T) = \sum_{e \in E_T} w(e).$$

Một đồ thị có thể có nhiều cây khung; cây khung có tổng trọng số nhỏ nhất được gọi là cây khung nhỏ nhất.

3.1.2 Cây khung nhỏ nhất đa mục tiêu

Khi mỗi cạnh trong đồ thị có nhiều trọng số, bài toán tương ứng được gọi là bài toán cây khung nhỏ nhất đa mục tiêu. Mục tiêu của lớp bài toán này là tìm tất cả cây khung tối ưu không bị trội. Ngay cả khi không thêm ràng buộc nào, lớp bài toán này vẫn thuộc nhóm NP-khó.

Cho một đồ thị vô hướng liên thông $G = (V, E)$, trong đó

$$V = \{v_1, v_2, \dots, v_n\}$$

là tập đỉnh hữu hạn. Với cạnh $e_{ij} \in E$, giả sử cạnh này có m thuộc tính dương

$$c(e_{ij}) = (c_{ij}^1, c_{ij}^2, \dots, c_{ij}^m),$$

trong đó mỗi thuộc tính có thể là khoảng cách, chi phí, độ khó, v.v. Dùng biến $x_{ij} \in \{0, 1\}$ để biểu diễn việc chọn cạnh e_{ij} . Một vector x biểu diễn một cây khung nếu các cạnh có $x_{ij} = 1$ tạo thành một cây trên V . Với mục tiêu thứ k , chi phí của cây là

$$f_k(x) = \sum_{e_{ij} \in E} c_{ij}^k x_{ij}.$$

Bài toán đa mục tiêu có thể viết là

$$\min_{x \in X} F(x) = \min_{x \in X} (f_1(x), f_2(x), \dots, f_m(x)),$$

trong đó X là tập tất cả cây khung khả thi. Ta cần tìm tập nghiệm không bị trội: không tồn tại nghiệm khác tốt hơn hoặc bằng nó trên mọi mục tiêu và tốt hơn hẳn trên ít nhất một mục tiêu.

So với cây khung nhỏ nhất thông thường, bài toán chỉ khác ở chỗ hàm mục tiêu không còn đơn lẻ. Chính việc có nhiều mục tiêu, mà các mục tiêu này thường mâu thuẫn với nhau, làm cho ta không thể dùng thuật toán kinh điển bằng cách lần lượt chọn cạnh có trọng số nhỏ nhất. Nếu quy nhiều mục tiêu thành một mục tiêu rồi áp dụng thuật toán kinh điển, ta chỉ thu được một nghiệm chứ không phải một nhóm nghiệm tối ưu không bị trội; hơn nữa, bản thân việc chuyển đổi từ nhiều mục tiêu sang một mục tiêu cũng là một khó khăn đối với người ra quyết định.

3.2 Dùng thuật toán di truyền để giải bài toán

3.2.1 Thiết kế mã hóa

Cayley đã chứng minh rằng với một đồ thị đầy đủ G , số cây nối tất cả n đỉnh là n^{n-2} . Dựa trên đó, Prüfer xây dựng một song ánh giữa các cây này và các chuỗi độ dài $n - 2$ lấy giá trị trong n số. Nếu đánh số các

đỉnh của đồ thị đầy đủ từ 1 đến n , thì mỗi chuỗi độ dài $n - 2$ trên tập $\{1, 2, \dots, n\}$ tương ứng với đúng một cây khung.

Bài viết dùng mã Prüfer cho cây khung: mỗi cây tương ứng với một chuỗi số độ dài $n - 2$, và ngược lại mỗi chuỗi như vậy tương ứng với đúng một cây. Chứng minh chi tiết của phép mã hóa từ cây sang chuỗi và phép giải mã từ chuỗi sang cây có thể xem trong các tài liệu liên quan; ở đây chỉ mô tả ngắn gọn.

Quá trình mã hóa.

1. Ban đầu chuỗi mã rỗng.
2. Gọi j là lá có số hiệu nhỏ nhất trong cây.
3. Tìm đỉnh i duy nhất kề với j , thêm i vào cuối chuỗi mã.
4. Xóa j và cạnh nối i, j khỏi cây; lúc này cây còn $n - 1$ đỉnh.
5. Lặp ba bước trên cho tới khi cây chỉ còn một cạnh. Chuỗi thu được là mã Prüfer của cây.

Quá trình giải mã.

1. Gọi P là chuỗi mã, và gọi S là tập các đỉnh của đồ thị không xuất hiện trong P .
2. Gọi i là đỉnh nhỏ nhất trong S , còn j là phần tử đầu tiên bên trái của P . Thêm cạnh ij vào cây; sau đó xóa i khỏi S và xóa phần tử j khỏi đầu P . Nếu j không còn xuất hiện trong P , thêm j vào S .
3. Lặp bước trên cho tới khi P rỗng.
4. Khi P rỗng, trong S còn đúng hai đỉnh; thêm cạnh nối hai đỉnh này vào cây. Cây thu được chính là cây tương ứng với chuỗi P ban đầu.

Hình dưới minh họa một cây khung và mã Prüfer tương ứng. Khi mã hóa, ta lần lượt xóa lá nhỏ nhất: 1, 2, 3, 4, và ghi các đỉnh kề tương ứng 4, 4, 4, 5. Có thể thấy mã Prüfer là một biểu diễn hiệu quả của cây khung: nó cho phép sinh ngẫu nhiên một cây rất dễ, và thông tin ở từng vị trí trong chuỗi mã tương đối đồng đều, nên rất phù hợp với các thao tác di truyền.

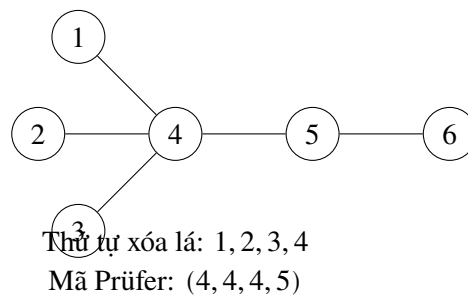


Figure 1: Một cây khung sáu đỉnh và chuỗi Prüfer thu được bằng quy tắc xóa lá nhỏ nhất.

3.2.2 Hàm đánh giá

Định nghĩa hàm đánh giá của nhiễm sắc thể x là

$$g(x) = \sum_{i=1}^m \left(\frac{\min[i]}{f_i(x)} \cdot 100 \right)^2,$$

trong đó $f_i(x)$ là chi phí của nhiệm sắc thể hiện tại trên mục tiêu thứ i , còn $\min[i]$ là chi phí nhỏ nhất trên mục tiêu thứ i trong tất cả nhiệm sắc thể đã sinh ra cho tới trước thể hệ hiện tại.

Cách định nghĩa này tốt hơn việc cộng trực tiếp trọng số trên các mục tiêu ở chỗ nó không chỉ phản ánh ưu thế của một nhiệm sắc thể trên từng mục tiêu, mà còn tránh được hiện tượng ưu thế ở một mục tiêu bị che lấp do các mục tiêu có miền giá trị hoặc xu hướng tổng thể khác nhau. Nhờ vậy thuật toán có thể thích ứng tốt hơn với các loại bài toán thực tế.

3.2.3 Định nghĩa thao tác di truyền

Toán tử lai ghép. Toán tử lai ghép được dùng là lai ghép theo đoạn nhỏ cùng vị trí: chọn ngẫu nhiên trong hai nhiệm sắc thể hai đoạn cùng vị trí, độ dài không quá 2, rồi hoán đổi chúng và chọn cá thể có độ thích nghi cao hơn để đưa vào thế hệ kế tiếp. Ví dụ tương ứng với hình 3-3 trong tài liệu gốc:

Parent 1	2	5	6	8	2	5
Parent 2	8	4	5	2	8	7
Offspring 1	2	5	5	2	2	5
Offspring 2	8	4	6	8	8	7

Toán tử đột biến. Toán tử đột biến dùng đột biến một điểm thông thường: sinh ngẫu nhiên một số từ 1 đến n để thay thế một vị trí nào đó trong chuỗi mã Prüfer. Ví dụ tương ứng với hình 3-4:

Parent	2	5	6	8	4	5
Offspring	2	5	6	2	4	5

Có thể thấy đột biến một điểm vẫn giữ lại phần lớn tính chất của nhiệm sắc thể ban đầu.

3.3 Thử nghiệm

Trong thử nghiệm, tác giả đặt tỷ lệ giữ lại là 54%, tỷ lệ lai ghép là 45%, tỷ lệ đột biến là 1%, đồng thời điều chỉnh số vòng lặp và kích thước quần thể theo từng bộ dữ liệu. Thuật toán được so sánh với tìm kiếm nguyên thủy. Để dễ quan sát mức xấp xỉ của thuật toán di truyền, mọi dữ liệu đều dùng hai mục tiêu.

Môi trường thử nghiệm: P4 2.0GHz, 256 MB DDR, Windows XP, Delphi 7.0.

3.3.1 Dữ liệu kinh điển quy mô nhỏ

Mã	Thông tin	Tìm kiếm: thời gian	Tìm kiếm: đúng	Tham số GA	GA: thời gian	GA: đúng
E1	$N = 3, K = 2, Ans = 2$	0s	hoàn toàn đúng	$L = 20, P = 10$	0s	hoàn toàn đúng
E2	$N = 4, K = 2, Ans = 6$	0s	hoàn toàn đúng	$L = 100, P = 30$	0s	hoàn toàn đúng
E3	$N = 5, K = 2, Ans = 12$	0s	hoàn toàn đúng	$L = 1000, P = 100$	2s	hoàn toàn đúng

3.3.2 Dữ liệu ngẫu nhiên quy mô vừa và lớn

Mã dữ liệu	H1	H2
Quy mô	$N = 10, K = 2$	$N = 11, K = 2$
Tìm kiếm nguyên thủy	22 phút	6 giờ 20 phút
Tham số GA	$L = 20000, P = 400$	$L = 30000, P = 400$
Thời gian GA	116s	296s
Độ đúng	Thu được một nhóm nghiệm rất gần nghiệm tối ưu; hình gốc minh họa phân bố nghiệm.	Thu được một nhóm nghiệm khá gần nghiệm tối ưu; hình gốc minh họa phân bố nghiệm.

3.3.3 Phân tích dữ liệu đặc biệt

Với tối ưu hóa hai mục tiêu, trong thực tế có thể tồn tại quan hệ giữa chi phí của hai mục tiêu. Ví dụ, khi các thiết bị tương tự nhau, một liên kết mạng có tốc độ tương đối cao thường có độ ổn định thấp hơn. Vì vậy tác giả thử nghiệm hai trường hợp đặc biệt: tương quan thuận và tương quan nghịch.

Tương quan thuận. Nếu với hai cạnh bất kỳ e_a, e_b của đồ thị, điều kiện

$$c_1(e_a) < c_1(e_b)$$

kéo theo

$$c_2(e_a) < c_2(e_b),$$

thì trọng số của đồ thị được gọi là tương quan thuận.

Tương quan nghịch. Tương tự, nếu với hai cạnh bất kỳ e_a, e_b , điều kiện

$$c_1(e_a) < c_1(e_b)$$

kéo theo

$$c_2(e_a) > c_2(e_b),$$

thì trọng số của đồ thị được gọi là tương quan nghịch.

Mã dữ liệu	S1	S2	S3
Quy mô	$N = 10, K = 2$	$N = 10, K = 2$	$N = 15, K = 2$
Đặc tính	tương quan thuận	tương quan nghịch	tương quan thuận
Tìm kiếm nguyên thủy	15 phút	28 phút	khoảng 600 năm
Tham số GA	$L = 2000, P = 400$	$L = 20000, P = 400$	$L = 20000, P = 400$
Thời gian GA	11 giây	304s	114 giây mỗi lần, chạy 15 lần
Độ đúng	Trong 5 lần chạy có xác suất 80%, tức 4 lần, thu được nghiệm tối ưu.	Thu được một nhóm nghiệm khá tương tự nghiệm tốt, xem hình minh họa trong tài liệu gốc.	Ở lần chạy thứ 12, chương trình thu được nghiệm tối ưu (3.11, 3.83). Đáng chú ý là lần thứ 3 đã có nghiệm xấp xỉ rất gần (3.14, 3.86), và ở lần thứ 6 cùng lần thứ 9 đều xuất hiện nghiệm khá tốt (3.18, 3.99).

Các biểu đồ phân tích dữ liệu H1, H2, S1, S2 và S3 so sánh phân bố nghiệm; nội dung định lượng chính của chúng được phản ánh trong các bảng trên.

4 Kết luận

Bài viết đã giới thiệu một số kiến thức cơ bản và kinh nghiệm sử dụng thuật toán di truyền, đồng thời thông qua kết quả thử nghiệm cho thấy thuật toán di truyền có ưu thế mạnh mà nhiều thuật toán khác khó sánh được khi giải các bài toán tối ưu tổ hợp. Đặc điểm của nó là có thể thu được nghiệm tương đối thỏa đáng trong thời gian ngắn, trong khi bản thân thuật toán khá đơn giản. Với rất nhiều bài toán thực tế mà thuật toán thông thường không giải quyết được, thuật toán di truyền có triển vọng ứng dụng tốt.

Thuật toán di truyền không chỉ là một thuật toán, mà còn là một cách tư duy. Trong tìm kiếm, vận dụng linh hoạt tư tưởng tiến hóa thường đem lại hiệu quả rất cao. Hiện nay thuật toán di truyền vẫn chưa thật phổ biến trong thi tin học; mục đích của bài viết là giúp ngày càng nhiều người yêu thích tin học hiểu thuật toán di truyền và tư tưởng của thuật toán tiến hóa.

Do năng lực của tác giả có hạn, bài viết khó tránh khỏi thiếu sót.

Tài liệu tham khảo

1. Chen Zhiping, Xu Zongben. *Toán học máy tính*. Science Press.
2. Zhou Ming, Sun Shudong. *Nguyên lý và ứng dụng của thuật toán di truyền*. National Defense Industry Press.
3. Shao Junli, Zhang Jing, Wei Changhua. *Cơ sở trí tuệ nhân tạo*. Electronics Industry Press.

A Chương trình nguồn

Phụ lục của tài liệu gốc đưa chương trình Pascal giải bài toán cây khung nhỏ nhất đa mục tiêu bằng thuật toán di truyền. Mã dưới đây giữ nguyên cấu trúc chính của chương trình, chỉ chuẩn hóa dấu nháy trong hai lệnh mở file để tránh ký tự Word cũ.

```
Program Ga;
const
    MaxN      =    50;
    MaxK      =     5;
    PopulationMax = 1000;
    Maxans    = 1000;
    Population =    40;
    Live      = 2000;
    Change    =    45;
    Suddenly  =     1;
type code=array[1..MaxN]of integer;
    vector=array[1..Maxk]of real;
    note=record
        v      :vector;
        data :code;
        power:longint;
    end;
    Group=array[1..PopulationMax]of note;
var
    inf,ouf:text;
```

```

d:array[1..MaxK,0..MaxN,0..MaxN]of real;
ans,ans2:array[1..Maxans]of vector;
ansb,ansb2:array[1..maxans]of code;
sn,n,k:longint; maxv:vector;
population,live,change,suddenly:longint;

function Genetic_Uncode(s:code):vector;
var temp:vector; uu:boolean;
    i,j,l,r:longint;
    s2:code; Se:set of 1..maxn;
function get:integer;
var i:longint;
begin
    for i:=1 to n do
        if i in Se then begin Se:=Se-[i]; get:=i; exit; end;
    end;
begin
    Se:=[1..n];
    for i:=1 to k do temp[i]:=0;
    for i:=1 to n-2 do Se:=Se-[s[i]];
    for i:=1 to n-2 do
        begin
            l:=get;
            for j:=1 to k do
                begin
                    if (l<=0)or(l>n) or (s[i]<=0)or(s[i]>n) then
                        begin
                            L:=L+1;
                        end;
                    temp[j]:=temp[j]+d[j,l,s[i]];
                end;
            uu:=true;
            for j:=i+1 to n-2 do
                if s[j]=s[i] then begin uu:=false; break; end;
            if uu then Se:=Se+[s[i]];
        end;
        l:=get; r:=get;
        for j:=1 to k do
            temp[j]:=temp[j]+d[j,l,r];
        Genetic_Uncode:=temp;
    end;

procedure Input_Initial;
var s,i,j:longint; tempcode:code;
begin
    readln(inf,n,k);
    for s:=1 to k do
        for i:=1 to n do

```

```

        for j:=1 to n do
            read(inf,d[s,i,j]);
        for i:=1 to n-2 do tempcode[i]:=random(n)+1;
        maxv:=Genetic_decode(tempcode); randomize;
        sn:=1; ans[1]:=maxv; ansb[1]:=tempcode;
        close(inf);
    end;

function find(v1,v2:vector):longint;
var i:longint;
    check:boolean;
begin
    check:=true;
    for i:=1 to k do
        if v1[i]<v2[i] then begin check:=false; break; end;
    if check then begin find:=1; exit; end;
    check:=true;
    for i:=1 to k do
        if v1[i]>v2[i] then begin check:=false; break; end;
    if check then begin find:=-1; exit; end;
    find:=0;
end;

procedure insert(v:vector; vcode:code);
var i,sn2:longint;
begin
    sn2:=0;
    if (v[1]=2) then begin end;
    for i:=1 to sn do
        case find(v,ans[i]) of
            0: begin inc(sn2); ans2[sn2]:=ans[i]; ansb2[sn2]:=ansb[i]; end;
            1: exit;
        end;
    inc(sn2); ans2[sn2]:=v; ansb2[sn2]:=vcode;
    sn:=sn2; ans:=ans2; ansb:=ansb2;
end;

procedure Genetic;
var S,S0:Group;
    i:longint; all:int64;

    procedure Genetic_Initial;
    var i,j:longint;
    begin
        for i:=1 to Population do
            for j:=1 to n-2 do
                s[i].data[j]:=random(n)+1;
            end;
        end;

```

```

function Genetic_Compute(v:vector; vcode:code):longint;
var i,mark:longint; nowv:vector;
begin
  nowv:=maxv; mark:=0; insert(v,vcode);
  for i:=1 to k do
  begin
    mark:=mark+sqr(round((nowv[i]/v[i])*100));
    if v[i]<maxv[i] then maxv[i]:=v[i];
  end;
  Genetic_Compute:=mark;
end;

procedure Genetic_Start;
var i:longint;
begin
  all:=0;
  for i:=1 to Population do
  begin
    s[i].v:=Genetic_Uncode(s[i].data);
    s[i].power:=Genetic_Compute(s[i].v,s[i].data);
    all:=all+s[i].power;
  end;
end;

function Genetic_Random:code;
var now:longint; seed:int64;
begin
  Seed:=random(all)+1; now:=1;
  while Seed>s[now].power do
  begin
    Seed:=Seed-s[now].power; inc(now);
  end;
  Genetic_Random:=s[now].data;
end;

function Genetic_Suddenly:code;
var temp:code;
  Seed:longint;
begin
  temp:=Genetic_Random; Seed:=random(N-2)+1;
  Temp[Seed]:=random(n)+1;
  Genetic_Suddenly:=temp;
end;

function Genetic_Change:code;
var temp1,temp2:code;
  Seed1,Seed2,temp:longint;

```

```

begin
    temp1:=Genetic_Random; Seed1:=random(N-2)+1;
    temp2:=Genetic_Random; Seed2:=random(N-2)+1;
    Temp:=temp1[Seed1];
    Temp1[Seed1]:=Temp2[Seed2];
    Temp2[Seed2]:=Temp;
    if Genetic_Compute(Genetic_Uncode(temp1),temp1)>
        Genetic_Compute(Genetic_Uncode(temp2),temp2) then
        Genetic_Change:=temp1
    else
        Genetic_Change:=temp2;
end;

procedure Genetic_Genarate;
var Seed,i:longint;
begin
    Genetic_Start;
    for i:=1 to Population do
    begin
        Seed:=random(100)+1;
        if Seed <= Suddenly then
            s0[i].data:=Genetic_Suddenly
        else if seed<= Change+Suddenly then
            s0[i].data:=Genetic_Change
        else
            s0[i].data:=Genetic_Random;
    end;
    s:=s0;
end;

begin
    Genetic_Initial;
    for i:=1 to Live do
    begin
        Genetic_Genarate;
    end;
end;

procedure sortans(l,r:longint);
var tl,tr:longint; tmp:vector;
begin
    tl:=l; tr:=r; tmp:=ans[l];
    while tl<tr do
    begin
        while (ans[tr][1]>tmp[1])and(tr>tl) do dec(tr);
        if tr=tl then break; ans[tl]:=ans[tr]; inc(tl);
        while (ans[tl][1]<tmp[1])and(tl<tr) do inc(tl);
        if tr=tl then break; ans[tr]:=ans[tl]; dec(tr);
    end;
end;

```

```

    end;
    ans[tl]:=tmp;
    if tl>l+1 then sortans(l,tl-1);
    if tr+1<r then sortans(tr+1,r);
end;

procedure Output_Result;
var i,j:longint;
begin
    sortans(1,sn);
    for i:=1 to sn do
    begin
        write(ouf,'The ',i,'th Answer:');
        for j:=1 to k do
            write(ouf,ans[i][j]:9:2);
        write(ouf,' Code=');
        for j:=1 to n-2 do write(ouf,' ',ansb[i][j]);
        writeln(ouf);
    end;
    writeln(ouf);
    for j:=1 to k do
    begin
        for i:=1 to sn do
            writeln(ouf,ans[i][j]:9:2);
        writeln(ouf,'-----');
    end;
    writeln(ouf,'Total=',sn);
    close(ouf);
end;

procedure start;
begin
    assign(inf,'input.txt'); reset(inf);
    assign(ouf,'edit8.text'); rewrite(ouf);
    Input_Initial;
    Genetic;
    Output_Result;
end;

begin
    start;
end.

```